

DESIGN AND IMPLEMENTATION OF A DATA BASE MANAGEMENT SYSTEM FOR Legal Case & Client Management System — Project Overview

NAME-GAGAN SRI VARMA ROLLNO.-25WU0102150



Objectives

Secure Client Storage Capture contact, identity and access metadata with role-based access controls.	Case Lifecycle Tracking Register cases, track status (Active, Pending, Closed), and maintain audit trails.	Hearing & Calendar Management Schedule hearings, set reminders, record outcomes and next steps.
Document & Evidence Management Reference and version-control case documents, with metadata for quick retrieval.	Advocate Assignment Link advocates to cases, capture roles (lead, co-counsel), and track workloads.	Reporting & Dashboards Generate lists of active cases, upcoming hearings and basic performance metrics.

Key Modules (High Level)



Client Management

Add, edit and view client profiles; contact history; confidentiality flags.



Case Management

Case registration, case number generation, type, filing date and status updates.



Hearing Management

Schedule hearings, log outcomes, create reminders and link hearing notes to cases.



Document Management

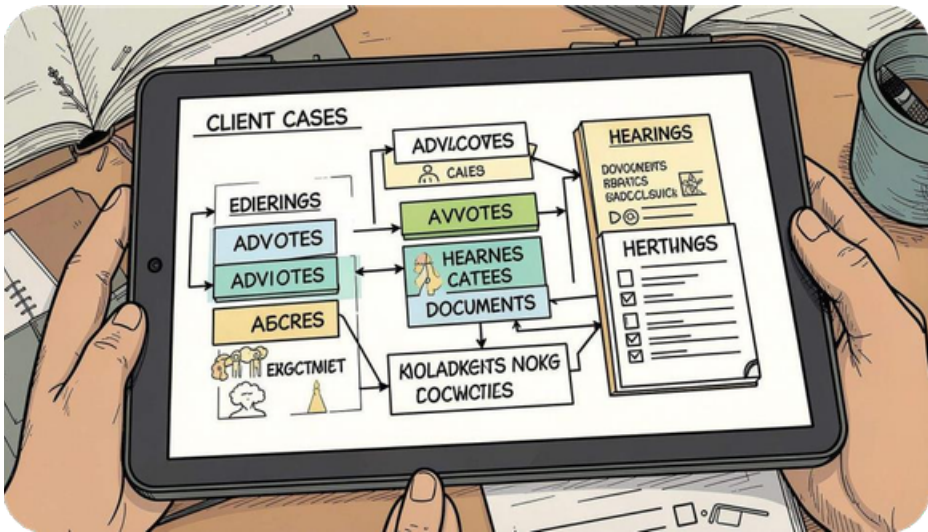
Store references to pleadings, evidence, scanned files and metadata (version, date).



Reports & Dashboard

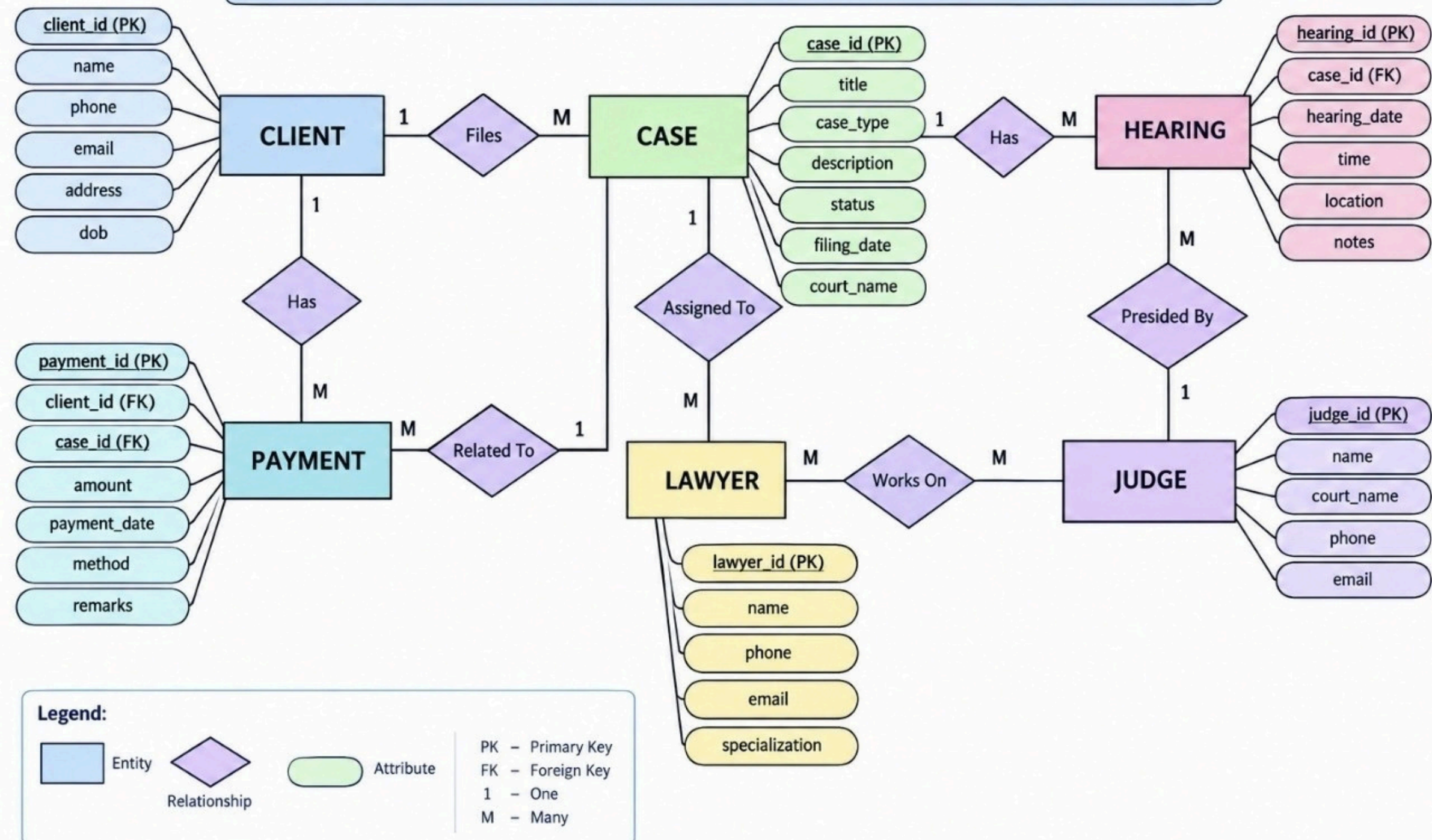
Active case lists, upcoming hearings, advocate workloads and exportable reports.

Database Design — Core Tables



- Main relational tables and purpose:
- Clients — personal & contact details, unique client_id
 - Cases — case metadata, case_number, client_id FK, status
 - Advocates — lawyer profiles, specialization, contact
 - CaseAdvocates — many-to-many: case_id + advocate_id, role
 - Hearings — hearing_date, court_name, outcome, linked to case_id
 - Documents — document_name, path/URI, uploaded_by, case_id
 - Users — application login, roles and permissions

ER Diagram – Legal Case & Client Management System



Sample Relational Schema & SQL Snippets

```
CREATE DATABASE LegalCaseDB;  
USE LegalCaseDB;
```

```
CREATE TABLE Clients (  
  client_id INT PRIMARY KEY AUTO_INCREMENT, name  
  VARCHAR(100) NOT NULL,  
  phone VARCHAR(15), email VARCHAR(100),  
  address VARCHAR(255)  
);
```

```
CREATE TABLE Advocates (  
  advocate_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  specialization VARCHAR(100), phone VARCHAR(15)  
);
```

```
CREATE TABLE Cases (  
  case_id INT PRIMARY KEY AUTO_INCREMENT,  
  client_id INT NOT NULL,  
  case_number VARCHAR(50) UNIQUE NOT NULL,  
  case_title VARCHAR(200) NOT NULL,  
  case_type VARCHAR(100), filing_date DATE,  
  status VARCHAR(30) DEFAULT 'Active',  
  FOREIGN KEY (client_id) REFERENCES Clients(client_id)  
);
```

These definitions form the foundation. Add indexes on case_number, hearing_date and foreign keys for performance. Implement soft-delete flags rather than hard deletes.

Example Queries & Reporting

Practical queries for legal workflows — concise, efficient and ready for dashboards.



Active Cases with Client

```
SELECT c.case_number,  
c.case_title, cl.name AS  
client_name, c.status FROM  
Cases c JOIN Clients cl ON  
c.client_id = cl.client_id WHERE  
c.status = 'Active';
```



Upcoming Hearings

```
SELECT h.hearing_date,  
h.court_name, c.case_number,  
cl.name FROM Hearings h JOIN  
Cases c ON h.case_id = c.case_id  
JOIN Clients cl ON c.client_id =  
cl.client_id WHERE  
h.hearing_date >= CURDATE()  
ORDER BY h.hearing_date;
```



Advocate Case Load

Aggregate cases per advocate
via CaseAdvocates to balance
assignments and plan
resourcing.

Implementation Roadmap & Technology Stack



Recommended stack and phased rollout:

Frontend: HTML, CSS, JavaScript (React or plain JS for

- lightweight UI)
- Backend: Python (Flask) or PHP — RESTful API to
- interact with DB
- Database: MySQL (ACID compliance), use InnoDB, enforce FK constraints
- Dev tools: VS Code, Git for source control, MySQL Workbench for schema design
- Security: TLS, hashed passwords, role-based access control, audit logging
- Deployment: Staging → UAT → Production; regular backups and migration scripts
-

❏ Next step: Prepare project synopsis, normalization notes, seed data, and a minimal frontend mock to demo workflows.